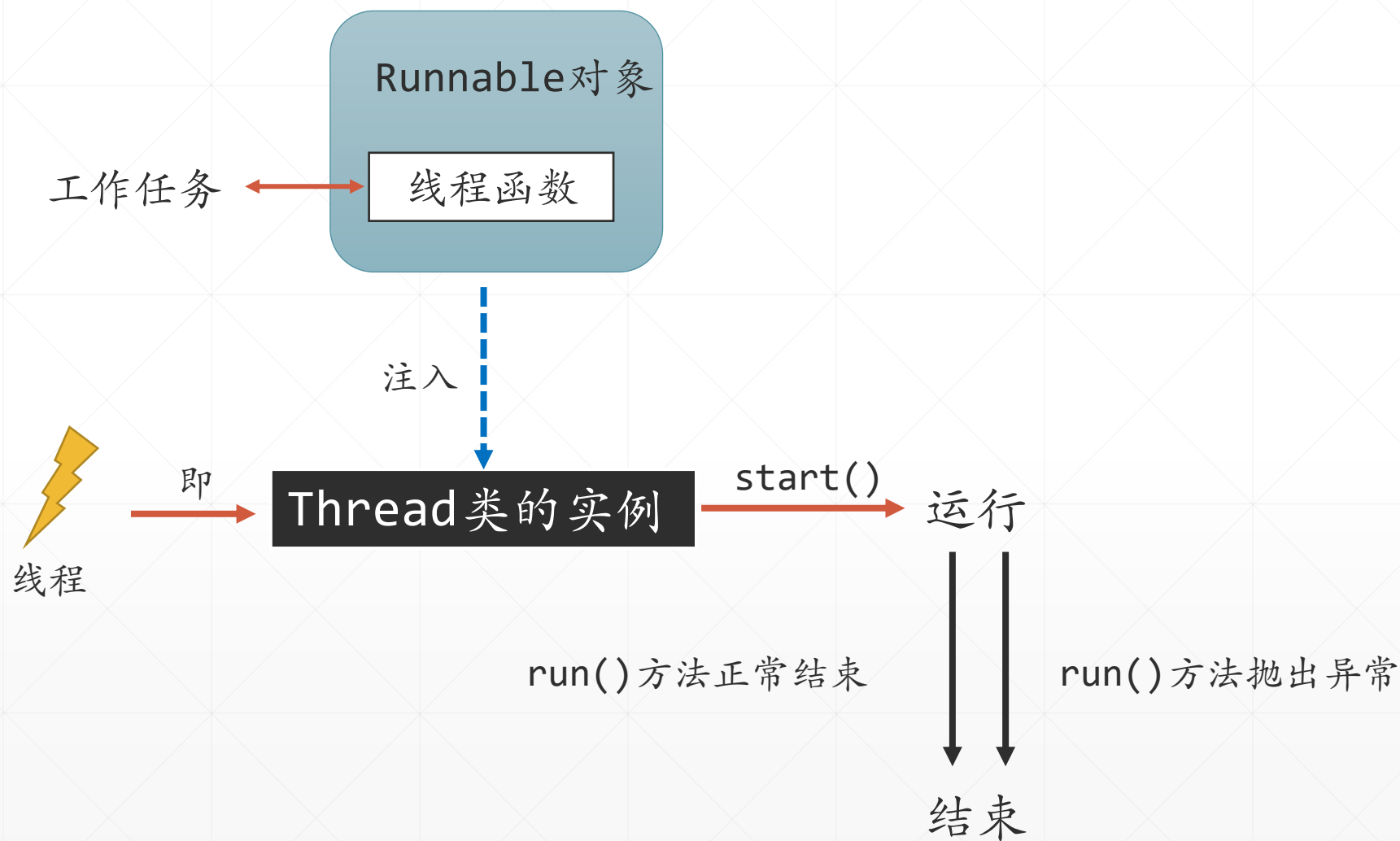




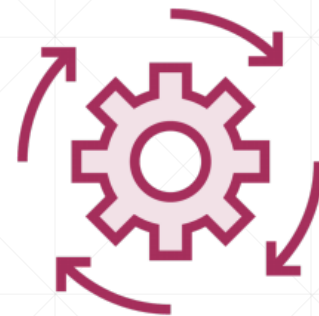
进一步了解线程对象

北京理工大学计算机学院
金旭亮

线程的运行



线程调度算法



线程只有获得了CPU的使用权，才能运行。
操作系统为特定的线程分配CPU的过程，称为“**线程调度**”。

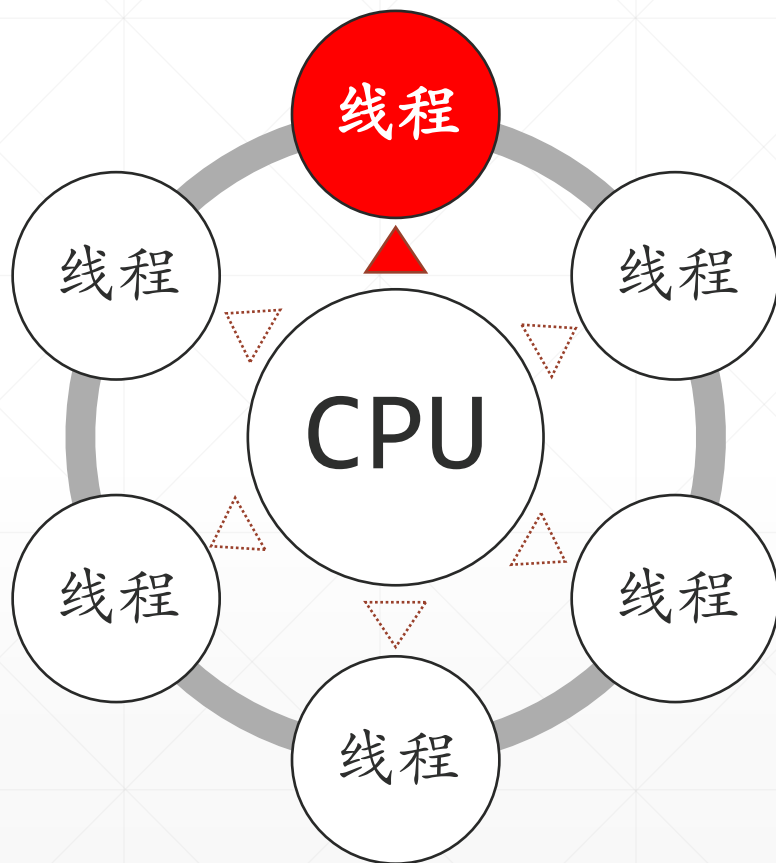
1

协作式调度算法：不会主动“剥夺”线程的CPU使用权，而由线程自己“主动让出”。Java中使用Thread类的yield()方法，可以主动让出CPU的使用权。现代操作系统，已经很少有采用这种调度算法的了。

2

抢占式调度算法：不允许单个线程长期独占使用CPU，操作系统统一考虑多种因素，负责选择合适的线程投入运行。这是当前操作系统普遍采用的线程调度算法。

基于“时间片”实现“抢占式线程调度算法”



现代操作系统，普遍使用“时间片”来给线程分配CPU，实现抢占式线程调度算法。

每个线程占用CPU的时间是有限的，最长就是一个“时间片”，“时间一到，必须让位”。

操作系统按照特定策略，从等待的线程中选择一个线程投入运行。

线程上下文切换



一个线程被暂停，剥夺CPU的使用权，然后，另一个线程被选中运行的过程，称为“**线程上下文切换**”。

“**线程上下文 (Thread Context)**”，指的是线程的运行环境，通常包括“通用寄存器”和“程序计数器”的内容。



多线程代码可能会比单线程代码慢，其中一个很重要的原因，在于过于频繁地进行线程上下文切换。因此，尽可能地减少不必要的线程上下文切换，是开发高性能应用的关键点。

线程优先级(Priority)



- ➔ 线程优先级会影响线程被操作系统选中分配CPU投入运行的概率。
- ➔ Java线程优先级的范围是`[1,10]`，默认值为5。
- ➔ 线程创建时，默认情况下，子线程继承父线程的优先级。
- ➔ 线程创建完毕后，可以通过调用`setPeriority`方法改变优先级。

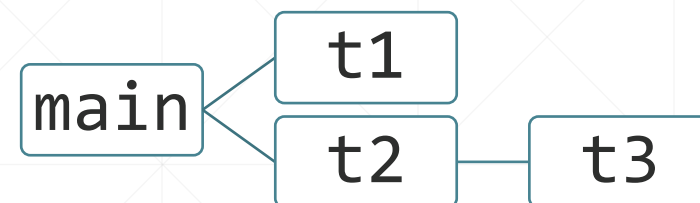


线程优先级仅供底层操作系统作为参考，只表明它有更多的机会获得CPU投入运行罢了，并不意味着高优先级的线程一定就比低优先级的线程先运行结束。

线程优先级示例

```
ThreadPriority.java x
1 public class ThreadPriority {
2     public static void main(String[] args) {
3         System.out.println("main的线程优先级: "
4             + Thread.currentThread().getPriority());
5         Thread t1 = new Thread();
6         System.out.println("t1的线程优先级: " + t1.getPriority());
7         Thread t2 = new Thread(() -> {
8             Thread t3 = new Thread();
9             System.out.println("t3的线程优先级: "
10                 + t3.getPriority());
11         });
12         t2.setPriority(6);
13         t2.start();
14         System.out.println("t2的线程优先级: " + t2.getPriority());
15     }
16 }
```

线程的父子关系

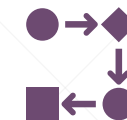


Run: ThreadPriority x

```
"C:\Program Files\Java
main的线程优先级: 5
t1的线程优先级: 5
t2的线程优先级: 6
t3的线程优先级: 6

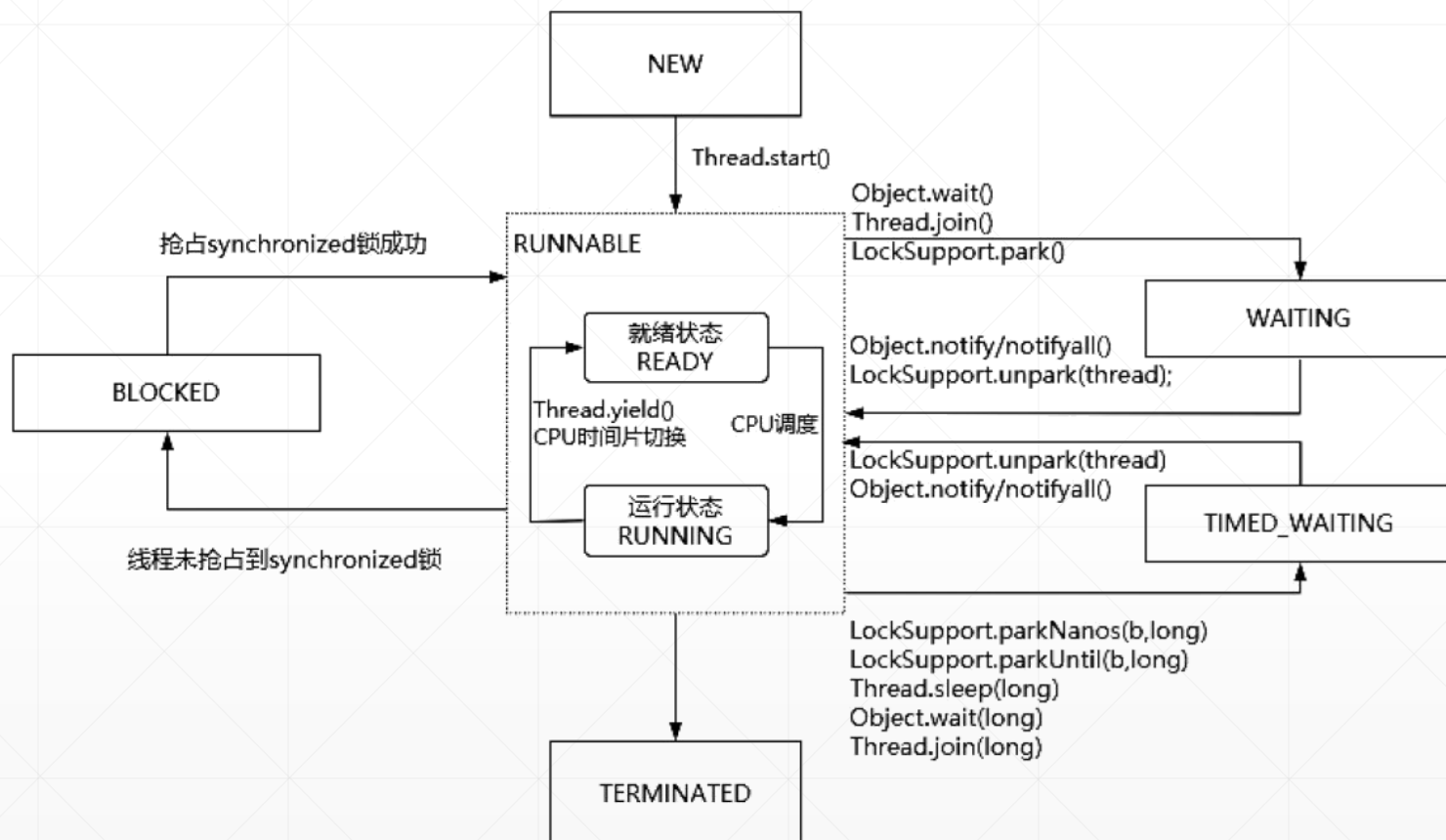
Process finished with
```

Java线程的六种状态



状态值	说明
NEW	线程已经创建完毕但还没开始执行
RUNNABLE	线程正在执行
BLOCKED	线程处于阻塞状态
WAITING	线程在等待另一个线程的通知
TIMED_WAITING	超时等待状态
TERMINATED	线程已经执行完毕

Java线程状态的切换



线程对象创建以后，调用不同的方法，线程对象会进入不同的状态。

这种相互转换状态图，称为“**状态机**”。

掌握“状态机”的面向对象实现方法，是一项基本编程技能。

线程状态示例

```
public class ThreadStateDemo {  
    public static void main(String[] args) throws InterruptedException {  
        //线程运行结束标记  
        AtomicBoolean finish = new AtomicBoolean( initialValue: false);  
        Thread thread = new Thread(() -> {  
            String currentThreadName = Thread.currentThread().getName();  
            System.out.println("工作线程:" + currentThreadName + "正在运行");  
            System.out.println("工作线程:"+currentThreadName + "开始休眠1秒。");  
            try {  
                TimeUnit.SECONDS.sleep( timeout: 1);  
            } catch (InterruptedException e) {  
                e.printStackTrace();  
            }  
            System.out.println("工作线程:"+currentThreadName + "结束休眠, 之后立即退出");  
            //设置线程结束标记  
            finish.set(true);  
        });
```

ThreadStateDemo.java

循环的判断条件，是一个可以跨线程安全访问的AtomicBoolean对象，初始值为false，当线程结束时，将其值置为true。

此示例启动了一个工作线程，让它休眠一秒之后退出。

主线程调用getState()方法循环查询工作线程的状态，跟踪其变化。

```
Thread.State state = thread.getState();  
System.out.println("工作线程状态: " + state);  
thread.start();  
while (!finish.get()) {  
    Thread.State currentState = thread.getState();  
    if (currentState != state) {  
        System.out.println("工作线程状态: " + currentState);  
        state = currentState;  
    }  
}  
thread.join();  
System.out.println("工作线程状态: " + thread.getState());  
}
```

示例运行截图



```
Run: ThreadStateDemo x
"C:\Program Files\Java\jdk-17\bin\java.exe"
工作线程状态: NEW
工作线程状态: RUNNABLE
工作线程: Thread-0正在运行
工作线程: Thread-0开始休眠1秒。
工作线程状态: TIMED_WAITING
工作线程状态: RUNNABLE
工作线程: Thread-0结束休眠, 之后立即退出
工作线程状态: TERMINATED

Process finished with exit code 0
```

除了getState()方法, Thread类中还提供有另外一些方法用于获取线程的当前状态, 比如isAlive()可用于测试线程是否处于活动状态(即线程已经启动且尚未终止)。请同学们自己编写代码测试一下这个方法的作用。



一个线程在其整个生命周期中, 只可能有一次处于NEW和TERMINATED状态。

查看JVM进程中的线程对象状态信息



```
PS C:\Users\JinXu> jps
```

```
1280 Main
```

```
16944 Jps
```

```
17184
```

```
15288 Launcher
```

```
13852
```

```
PS C:\Users\JinXu> jstack 1280
```

```
2021-09-22 22:49:45
```

```
Full thread dump Java HotSpot(TM) 64-Bit Server VM (17+35-LTS-2724 mixed mode, sharing):
```

```
Threads class SMR info:
```

```
_java_thread_list=0x000002112b44af70, length=18, elements={
```

```
0x000002117fed49a0, 0x000002117fed6230, 0x000002112b0060c0, 0x000002112b007890,
```

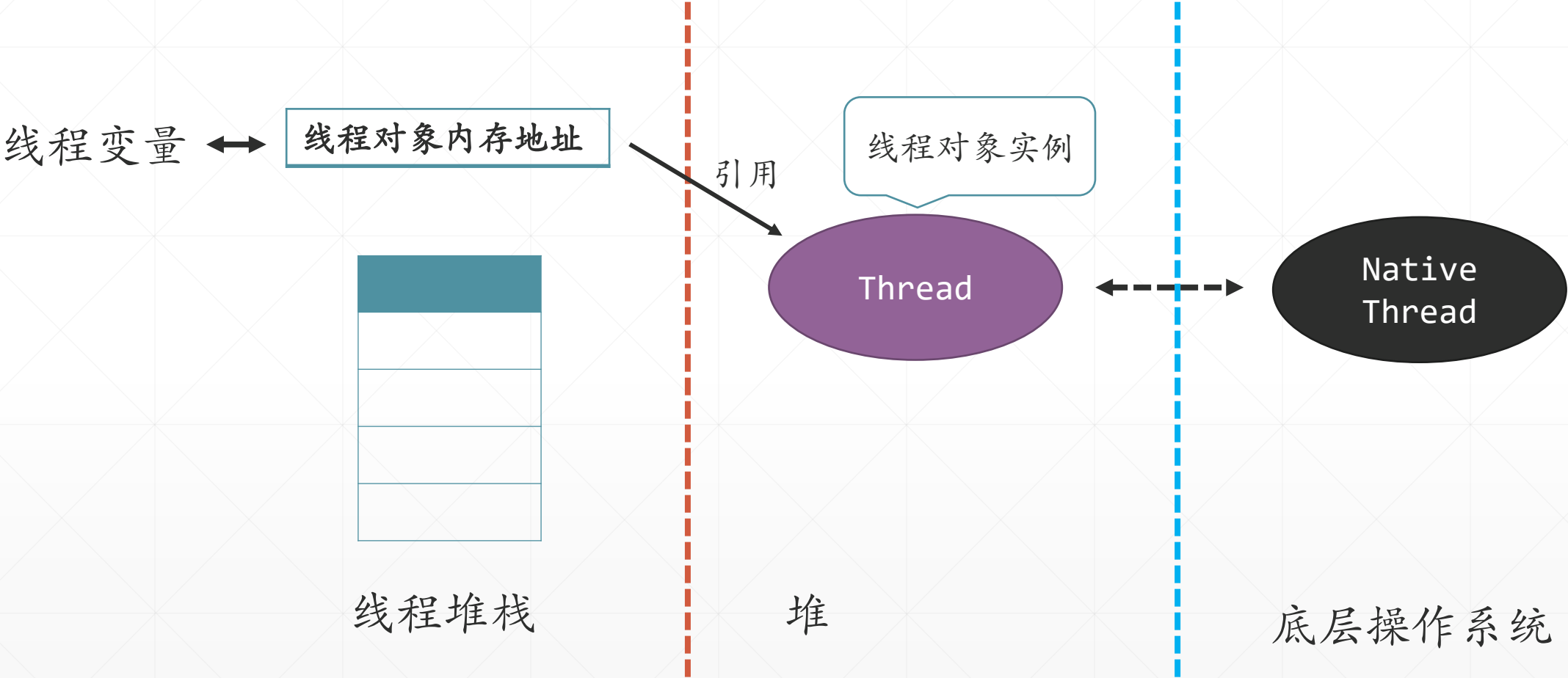
```
0x000002112b00a350, 0x000002112b00ed10, 0x000002112b013a00, 0x000002117fee8bc0,
```

```
0x000002117feeeac0, 0x000002112b108590, 0x000002112b10a270, 0x000002112b109da0,
```

```
0x000002112b10a740, 0x000002112b10ac10, 0x000002112b10b0e0, 0x000002112b1080c0,
```

当某个Java程序正在运行时，使用jps可以查看到此进程的id，使用此id值，用jstack命令，可以查看到选定进程中各个线程对象的相关信息。

线程对象的内存模型



关于线程堆栈



操作系统会为每一个线程都分配一个线程堆栈，那些保存于堆栈中的数据被划分为“**栈帧 (stack frame)**”，主要用于保存线程函数中定义的局部变量，以及方法调用相关的信息等。



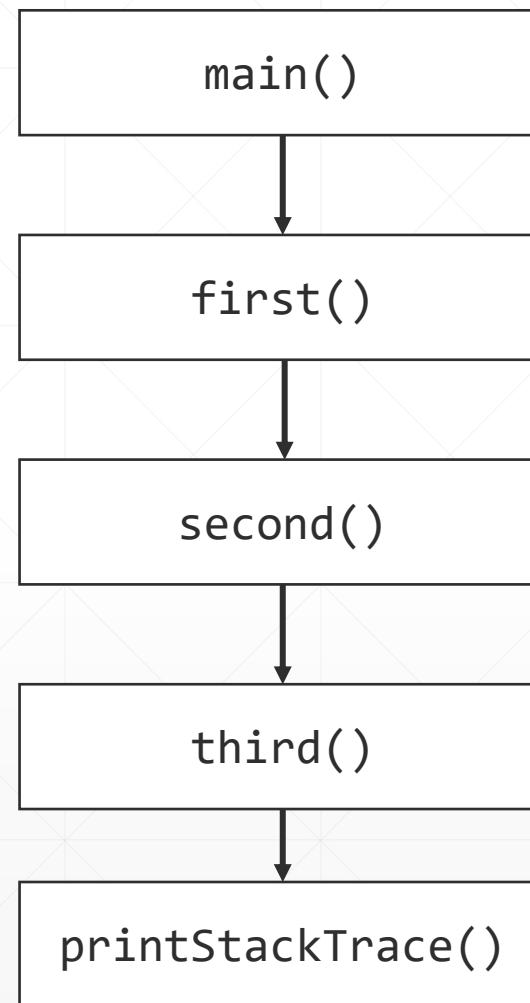
由于每一个线程都对应着一个线程堆栈，这个堆栈需要占用特定数量的内存空间，因此，一台计算机能够创建的线程个数是有限的，如果创建了太多的线程，计算机就会因为资源耗尽而失去响应。



线程函数可以调用其它的函数，这种调用对应着栈帧在栈中的push和pop操作。

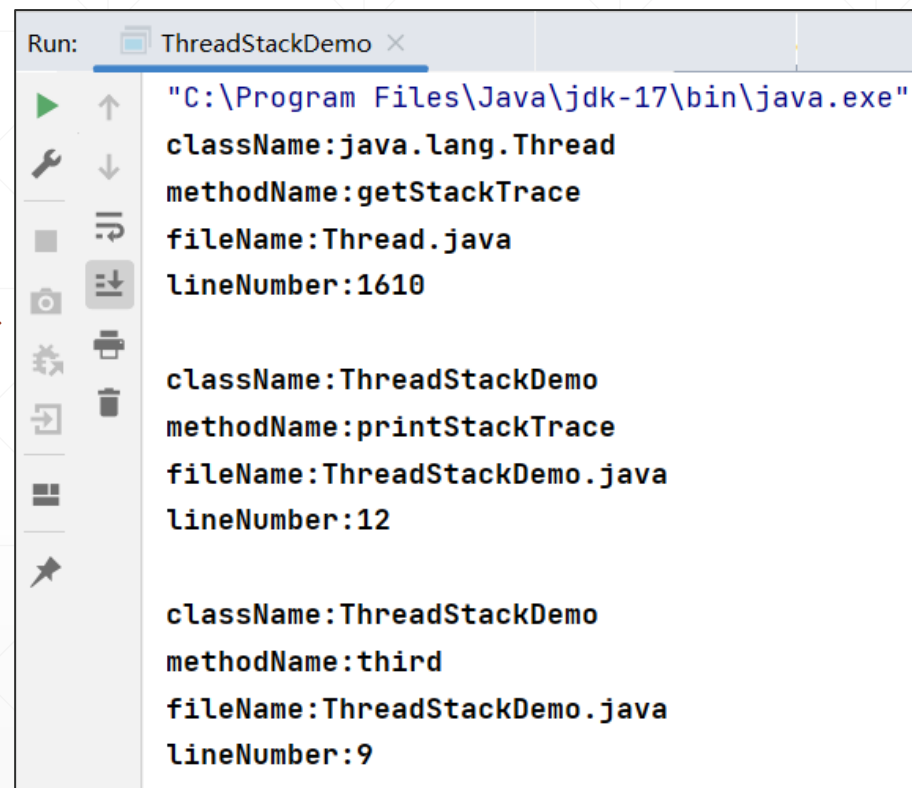
直观了解线程堆栈-1

```
public class ThreadStackDemo {  
    public void first(){  
        second();  
    }  
    1 usage  
    public void second(){  
        third();  
    }  
    1 usage  
    public void third(){  
        printStackTrace();  
    }  
    1 usage  
    public void printStackTrace(){...}  
  
    public static void main(String[] args) {  
        new ThreadStackDemo().first();  
    }  
}
```



直观了解线程堆栈-2

```
public void printStackTrace() {  
    //获取当前线程的堆栈信息  
    var array : StackTraceElement[] = Thread.currentThread().getStackTrace();  
    for (StackTraceElement elem : array) {  
        System.out.println("className:" + elem.getClassName() + "\n"  
            + "methodName:" + elem.getMethodName() + "\n"  
            + "fileName:" + elem.getFileName() + "\n"  
            + "lineNumber:" + elem.getLineNumber() + "\n");  
    }  
}
```



部分截图

堆栈溢出



```
StackOverflow.java x
1 usage
1 public class StackOverflow {
2     public static void main(String[] args) {
3         var thread = new Thread(StackOverflow::doSomethingBad);
4         thread.start();
5     }
6
7     //构造一个无限递归的函数
8     static void doSomethingBad() {
9         doSomethingBad();
10    }
11 }
```

由于线程堆栈的容量是有限的，因此，如果函数调用的次数过多，就很容易耗尽容量，引发“堆栈溢出”错误。

[illegible]